노이즈 마진

수식	의미
V_NML = V_IL - V_OL	Low 노이즈 마진
V_NMH = V_OH - V_IH	High 노이즈 마진

Number Systems

수식	의미
2’s Comp(N) = 비트반전 + 1	2의 보수 변환
범위 (n비트 2’s Comp) = -2^(n-1) ~ +2^(n-1)-1	표현 범위
Overflow = Cin(MSB) XOR Cout(MSB)	오버플로우 검출
Fixed-point 변환: 실수 × 2^f → 반올림 → 이진수	고정소수점 변환
Floating-point: M × 2^(E-Bias)	부동소수점 값
절대 오차 (f비트 Fixed) = 2^(-(f+1))	고정소수점 최대 오차

FA / Adders

수식	의미
S = A ⊕ B ⊕ CI	FA 합(Sum)
CO = AB + B·CI + CI·A	FA 올림수(Carry-out)
A - B = A + B’ + 1	2의 보수 뺄셈 (Cin=1, B 비트반전)
RCA: Area=O(N), Delay=O(N)	N비트 Ripple Carry Adder
CSA: Area=O(N), Delay=O(1)	N비트 Carry Save Adder

P&G Based Adders

수식	의미
Gi = Ai · Bi	Carry Generate (A=B=1이면 무조건 올림수)
Pi = Ai ⊕ Bi	Carry Propagate (Cin을 전달)
Ci+1 = Gi + Pi·Ci	Carry 점화식
Si = Pi ⊕ Ci	Sum
G(i:j) = G(i:k) + P(i:k)·G(k-1:j)	그룹 Generate 결합
P(i:j) = P(i:k) · P(k-1:j)	그룹 Propagate 결합
CLA C4 = G3+P3G2+P3P2G1+P3P2P1G0+P3P2P1P0C0	4비트 CLA 전개
PPA: Delay = O(log N)	Parallel Prefix Adder
Gi = Ai·Bi	Generate (자체 올림수 생성)
Pi = Ai ⊕ Bi	Propagate (올림수 전파)
	P&G로 재표현한

Ci+1 = Gi + Pi·Ci	Carry
Si = Pi ⊕ Ci	P&G로 재표현한 Sum
G_{i:j} = G_{i:k} + P_{i:k}·G_{k-1:j}	Group Generate (블록 결합)
P_{i:j} = P_{i:k} · P_{k-1:j}	Group Propagate (블록 결합)
PPA: Area=O(NlogN), Delay=O(logN)	Parallel Prefix Adder

타이밍 제약 (Timing Constraints)

수식	조건	비고
t_cy > t_dCQ + t_dMax + t_s	Setup (Skew 없음)	위반 시 클럭 늘리기
t_h < t_cCQ + t_cMin	Hold (Skew 없음)	"Unsafe at any speed"
t_cy > t_dCQ + t_dMax + t_s - t_k	Setup (Skew 포함)	t_k↑ → Setup 유리
t_h < t_cCQ + t_cMin + t_k	Hold (Skew 포함)	t_k↑ → Hold 불리
Slack = t_cy - (t_dCQ + t_dMax + t_s)	Setup Slack	>0이면 통과
t_cy_min = t_dCQ + t_dMax + t_s - t_k	최소 클럭 주기	
f_max = 1 / t_cy_min	최대 동작 주파수	

Metastability / Synchronizer

수식	의미
DV(t) = DV(0) × e^(t/τ)	메타스태블 상태 해소 동역학
P_E = (t_s + t_h) / t_cy	메타스태블 진입 확률
P_S = exp(-t_w / τ)	탈출 못할 확률
P_F = P_E × P_S	단일 이벤트 오류 확률
f_F = f_a × P_F	단위 시간당 오류 빈도
MTTF = 1 / f_F	평균 고장 간격
t_w = t_cy - t_s - t_dCQ	FF 2개 기준 대기 시간

저전력 설계

수식	의미
P_dynamic = ½ × S × C × V^2 × F	동적 전력
I_sub = K_1×W×e^(-Vth/nV_0)	Subthreshold Leakage
I_ox = K_2×W×(V/T_ox)^2×e^(-aT_ox)	Gate-oxide Leakage

Combinational vs Sequential

구분	Combinational	Sequential

출력	f(현재 입력)	f(현재 입력 + 이전 상태)
메모리	없음	있음 (레지스터)
클럭	불필요	필요
피드백	없음	있음
Verilog	assign / always @(*)	always @(posedge clk)

Latch vs Flip-flop

구분	Latch	Flip-flop
트리거 방식	Level-sensitive	Edge-triggered
언제 바뀌나	Enable=1인 동안 계속	클럭 엣지 순간만
투명성	있음 (Transparent)	없음
용도	특수 상황	일반 순차 논리 (표준)

Setup vs Hold Violation

구분	Setup Violation	Hold Violation
원인	데이터가 너무 늦게 도착	데이터가 너무 빨리 도착
클럭으로 해결?	✅ 클럭 주기 늘리기	❌ "Unsafe at any speed"
해결책	t_cy↑, Critical Path 최적화	버퍼 삽입 (t_cMin↑)
관련 지연	t_dMax (최악 지연)	t_cMin (최선 지연)

Propagation vs Contamination Delay

구분	Propagation Delay (t_d)	Contamination Delay (t_c)
정의	입력 안정 → 출력 안정	입력 첫 변화 → 출력 첫 변화
경우	최악 (Worst case)	최선 (Best case)
용도	Setup Constraint	Hold Constraint
대소	t_d ≥ t_c	t_c ≤ t_d

Static vs Dynamic Power

구분	Static Power	Dynamic Power
발생 시점	항상 (동작 무관)	회로 동작 시
주요 원인	Leakage 전류	커패시턴스 충방전
민감 요소	Vth, T_ox	V^2, F
공정 미세화 영향	기하급수적 증가	증가
해결 기법	Power Gating, High-k	Clock Gating, DVS

Clock Gating 비교

구분	ILD (Idle Logic Driven)	DTD (Data Toggling Driven)
기반	en 신호	XOR (din ≠ dout 검출)
적용 범위	en 있는 FF만	모든 FF (범용)
면적 overhead	작음	40%↑

클럭 절감	~37%	~33%

로직 표현 5가지 유일성

표현	유일성	주요 용도
English	✗	스펙 작성
Boolean Expression	✗	수학적 분석
Truth Table	✓	완전한 함수 정의, 검증
Schematic	✗	실제 구현
BDD (순서 고정 시)	✓	EDA 툴, 등가 검증

Binary vs One-Hot State Assignment

구분	**Binary** 인코딩	**One-Hot** 인코딩
비트 수	$\lceil \log_2 n \rceil$ 비트	n 비트
FF 수	적음 (최소)	많음 (상태 수만큼)
조합 논리	복잡	단순
속도	보통	빠름
면적	작음	큼
next state 수식	카르노맵 필요	전이도에서 직접 읽음
주요 사용처	상태 많은 FSM, ASIC	상태 적은 FSM, FPGA

Moore vs Mealy vs Synchronous Mealy

구분	**Moore**	**Mealy**	**Synchronous Mealy**
출력 결정	f(state)	f(state, input)	f(state, input)
출력 동기화	비동기 (조합)	비동기 (즉각)	동기 (FF 거침) ✓
글리치	없음 ✓	있음	없음 ✓
상태 수	많음	적음	적음
변환	—	—	Output-oriented로 Moore→변환 가능

Sequential vs Parallel Multiplier

구분	**Sequential**	**Parallel**
회로 크기	작음	큼
소요 시간	많이 걸림	적게 걸림
구조	반복 shift+add	AND배열+CSA트리+Final Adder

상수 곱셈 최적화 방법

방법	적용 조건	변환 예시
Shift-and-Add	1이 적게 분포	a×36 = (a<<5)+(a<<2)
Canonical Encoding	1이 연속 분포	a×62 = (a<<6)-(a<<1)

Verilog 핵심 구분

구분	wire	reg
의미	물리적 연결선	값 저장 변수
사용 위치	assign, 포트 연결	always 블록 내부
회로 대응	조합 논리	FF or 조합

Verification vs Test vs Characterization

구분	Verification	Test	Characterization
목적	설계가 스펙 만족?	제조가 올바른가?	성능 파라미터 측정
시점	제조 전	제조 후	제조 후
대상	설계 자체	모든 칩 (전수)	샘플만
주요 도구	Simulation, STA, Formal	ATPG, Scan, BIST	V-I 측정, 가속 수명

정수 표현 방식 비교 (n=4비트 기준)

구분	Sign-Magnitude	1's Complement	2's Complement
음수 변환	MSB=1, 절대값	비트 전체 반전	비트 반전 + 1
0의 개수	2개 (+0, -0)	2개 (+0, -0)	1개 ✅
표현 범위	-7 ~ +7	-7 ~ +7	-8 ~ +7 ✅
덧셈 복잡도	복잡	중간 (End-around carry)	단순 ✅
현재 사용	거의 없음	없음	현대 표준 ✅

Fixed-Point vs Floating-Point

구분	Fixed-Point	Floating-Point
소수점 위치	고정	동적 (이동 가능)
표현 범위	좁음	매우 넓음
오차 종류	절대 오차	상대 오차
연산 복잡도	단순	복잡 (FPU 필요)
음수 방식	2의 보수	부호-절대값 (MSB)
표기법	i.f, si.f	M × 2^(E-Bias)
주요 용도	DSP, 임베디드	과학 계산, 그래픽

Combinational Building Blocks 비교

블록	입력	출력	기능	Verilog 핵심
Decoder	n비트 이진	2^n 비트 One-hot	이진 →One-hot	`1 << a`

Encoder	2^n 비트 One-hot	n비트 이진	One-hot→이진	OR 패턴
MUX	n개 k비트 입력 + select	k비트	입력 선택	`({k{s[i]}} & ai)` OR
Arbiter	n비트 request	n비트 grant	첫 번째 1 선택	carry chain
Priority Encoder	n비트	$\log_2 n$ 비트	첫 1의 위치 이진수	Arbiter+Encoder
Eq Comparator	a, b (n비트)	1비트 eq	a==b?	`a==b`
Mag Comparator	a, b (n비트)	1비트 gt	a>b?	`a>b`

ROM vs RAM vs PLA

구분	ROM	RAM	PLA
쓰기 가능	✗	✓	✗ (설계 시 고정)
구조	Decoder + 연결망	Decoder + R/W	AND배열 + OR배열
AND항 수	2^n (모든 Minterm)	2^n	필요한 것만
주요 용도	룩업 테이블, 로직	데이터 저장	복수 SoP 함수

RCA vs CSA

구분	RCA (Ripple Carry Adder)	CSA (Carry Save Adder)
구조	FA 직렬 연결	FA 배열, CO 저장
Carry 전파	LSB→MSB 순차 전파	전파 없음 (저장)
Area	O(N)	O(N)
Delay	O(N) — carry ripple	O(1) — 병렬 처리
출력 형태	완전한 2진수	Redundant (S줄+C줄)
마무리	불필요	마지막에 CLA 한 번 필요
용도	단순 2수 덧셈	다수 수의 합 (곱셈 내부 등)

Adder 전체 비교 (P&G Based)

구분	RCA	CLA	PPA	Carry-Skip	Carry-Select
Delay	O(N)	O(1)*	O(log N)	O(√N)	O(√N)
Area	O(N)	O(N)	O(N log N)	O(N)	O(N)
구조	FA 직렬	2단 AND-OR	Prefix Tree	블록+Skip MUX	블록×2+MUX
특징	단순	fanin↑	빠름, 배선복잡	면적 효율	속도/면적 균형

*CLA: fanin 급증으로 소규모에만 실용적

Adder 종류 종합 비교

종류	핵심 아이디어	Area	Delay
RCA	FA 직렬, Carry 순차 전파	O(N)	O(N)
CLA	Carry 직접 계산 (2단 AND-OR)	O(N)	O(1)*
PPA (Prefix)	블록 결합 트리	O(NlogN)	O(logN)
Carry-Skip	블록 P=1이면 Skip	O(N)	O(√N)
Carry-Select	Cin=0/1 미리 계산 + MUX	O(N)	O(√N)
CSA	CO 저장, 3→2 압축	O(N)	O(1)

*CLA: 팬인 제한으로 4~8비트 블록에서 실용적

Stuck-at Fault 검출 조건

조건	설명
Sensitization	해당 노드가 결함값과 반대 값이 되도록 입력 인가
Propagation	해당 노드의 값이 외부 출력까지 전달되도록 설정
Fault Coverage	검출된 Fault 수 / 전체 Fault 수 (목표: 95~99%)

2's Complement (2의 보수): 현대 컴퓨터 정수 표현 표준. 비트 반전+1. 0 하나, 덧셈 단순. n비트 범위: -2^(n-1) ~ 2^(n-1)-1.

Arbiter (중재기): n비트 request 중 첫 번째 1을 찾아 grant. carry chain 구조. Priority Encoder = Arbiter + Encoder.

ATPG (Automatic Test Pattern Generation): 특정 Fault를 검출하는 테스트 패턴을 자동 생성. Sensitization + Propagation 조건 만족.

BDD (Binary Decision Diagram): 변수 순서 고정 후 결정 트리를 압축한 논리 함수 표현. ROBDD는 유일성 보장.

BIST (Built-In Self Test): 칩 내부에 테스트 회로 내장. LFSR(패턴 생성) + MISR(결과 압축)으로 자가진단.

Canonical SoP: 모든 항이 모든 변수를 포함하는 SoP. 진리표와 1:1 대응, 유일성 보장.

CDN (Clock Distribution Network): 클럭을 전체 FF에 동시 공급하는 배선망. H-Tree 구조, Clock Skew 최소화 목표.

Clock Skew (t_k): FF2 클럭 도착 시간 - FF1 클럭 도착 시간. Setup에는 유리, Hold에는 불리 (양수일 때).

Contamination Delay (t_c): 입력이 처음 변한 순간 → 출력이 처음 흔들리기 시작하는 시간. Hold에 사용.

Arbiter (중재기): n비트 request 중 첫 번째 1을 찾아 grant. carry chain 구조. Priority Encoder = Arbiter + Encoder.

Critical Path: 칩에서 가장 지연이 긴 신호 경로. 최대 동작 주파수를 결정.

Carry-Select Adder: 상위 블록을 Cin=0/1 두 경우로 미리 계산 후 MUX로 선택. Delay ≈ O(√N).

Carry-Skip Adder: 블록 내 P_{block}=1이면 Carry가 블록을 건너뜀. MUX로 Skip 경로 선택.

Canonical Encoding: 연속된 1의 블록을 (상위+1) - 하위로 변환해 1의 개수 감소. 상수 곱셈 최적화에 사용.

Carry-Select Adder: 상위 블록을 Cin=0, Cin=1 두 경우 동시 계산 → Cin 확정 시 MUX 선택. Delay=O(√N).

Carry-Skip Adder: P(i-1:k)=1이면 Cin이 블록 전체 Skip. Delay=O(√N). 면적 효율적.

Characterization (특성화): 샘플 칩의 동작 범위·파라미터·노화 특성 측정. NBTI, HCI, 온도 영향 포함.

CLA (Carry Look-ahead Adder): Carry를 원래 입력(P, G)에서 직접 계산. 2단 AND-OR 논리. Delay=O(1) 이론적, fanin 증가 한계.

CLA (Carry Look-ahead Adder): P&G로 Carry를 중간 단계 없이 직접 2단 AND-OR로 계산. 팬인 제한으로 4~8비트 블록에서 실용적.

CSA (Carry-Save Adder): 3개 입력을 CO 전파 없이 (S줄, C줄)로 압축. Area=O(N), Delay=O(1). 마지막에 CLA로 합산.

Decoder (디코더): n비트 이진 → 2^n 비트 One-hot 변환. b=1<<a. 분할 정복으로 대형 디코더 구성.

DFT (Design For Testability): 테스트 용이성을 위한 설계 방법론. Controllability + Observability 향상.

DVS (Dynamic Voltage Scaling): 부하에 따라 전압/주파수를 동적으로 조절. DC-DC Converter 필요.

Encoder (인코더): One-hot → 이진 코드 변환. Decoder 역방향. 대형: 2단 구성 + any-input-true 출력.

Essential PI: 특정 Minterm을 오직 하나의 PI로만 커버할 수 있는 Prime Implicant. 반드시 선택.

FA (Full Adder, 전가산기): 3비트(A,B,CI) 입력 → S=A⊕B⊕CI, CO=AB+B·CI+CI·A. “(3,2) counter”라고도 불림.

Fault Coverage: 검출된 Stuck-at Fault 수 / 전체 Fault 수. 테스트 품질 지표. 목표 95~99%.

FIFO Synchronizer: 멀티비트 신호를 클럭 도메인 간 안전하게 전달. 포인터만 Gray Code + Brute-Force Sync.

Fixed-Point (고정소수점): si.f 형식. 소수점 위치 고정. 절대 오차 = 2^(-(f+1)). 변환: 실수×2^f → 반올림 → 이진수.

Floating-Point (부동소수점): M × 2^(E-Bias). Mantissa(가수) + Exponent(지수). Implied 1 (정규화 시 맨 앞 1 저장 생략). 상대 오차.

Formal Verification: 수학적 증명으로 기능 정확성 확인. 테스트 패턴 불필요. 소규모 블록에 적합.

MUX (Multiplexer): n개의 k비트 입력 중 select로 하나 선택. One-hot select: s[i]로 마스킹 후 OR. Binary MUX = Decoder + One-hot MUX.

Overflow (오버플로우): 2의 보수 연산 결과가 표현 범위 초과. 검출: Carry-in(MSB) XOR Carry-out(MSB) = 1이면 오버플로우.

PLA (Programmable Logic Array): AND 배열 + OR 배열. ROM보다 적은 AND항으로 복수 SoP 함수 구현.

PPA (Parallel Prefix Adder): 3단계(Pre-processing→Prefix Tree→Post-processing). G_{i:-1}을 트리로 계산. Area=O(NlogN), Delay=O(logN). Kogge-Stone, Brent-Kung 등 변형 존재.

RCA (Ripple Carry Adder): FA N개 직렬 연결. Carry가 LSB→MSB 순차 전파. Area=O(N), Delay=O(N).

Gray Code: 인접 값 사이에 1비트만 변화하는 인코딩. 멀티비트 동기화에 사용.

Hold Time (t_h): 클럭 엣지 이후 D 입력이 안정적으로 유지되어야 하는 최소 시간.

Implicant: 출력=1을 만드는 입력 조합 집합. k-cube = 2^k개 Minterm 묶음.

IR Drop: 전원 배선의 저항으로 인한 전압 강하. Power Integrity 문제.

Implication Chart Method: 두 상태를 합칠 수 있는지 체계적으로 판별하는 State Minimization 기법. 출력이 다른 쌍은 X, 조건 실패 시 연쇄적으로 X 처리.

Mealy Machine: 출력 = f(state, input). 전이 화살표에 출력 표기. 즉각 반응, 상태 수 적음. 글리치 가능.

Metastability: Setup/Hold 위반 시 FF가 0도 1도 아닌 불법 중간 상태에 빠지는 현상. 제거 불가, 확률만 줄일 수 있음.

Moore Machine: 출력 = f(state). 상태 원 안에 출력 표기. 클럭 동기 출력, 글리치 없음. 신호등 FSM이 대표 예.

Synchronous Mealy FSM: Mealy 출력을 FF를 통해 동기화. 비동기 출력 문제 해결. Output-oriented Assignment로 Moore에서 변환 가능.

Minterm (m_n): 모든 변수가 정확히 한 번 등장하는 AND항. n번 Minterm = 이진수 n에 해당하는 입력 조합.

PDN (Power Delivery Network): 전원을 칩 전체에 공급하는 배선망. Grid 구조, IR Drop 최소화.

PPA: Power / Performance / Area. 설계의 3대 지표, 항상 트레이드오프.

Prime Implicant (PI): 더 이상 크게 묶을 수 없는 최대 크기 Implicant.

Propagation Delay (t_d): 입력이 완전히 안정된 후 → 출력이 완전히 안정될 때까지의 시간. Setup에 사용.

RTL (Register Transfer Level): 클럭 단위로 레지스터 간 데이터 전송을 기술하는 설계 수준.

Scan: FF들을 직렬 체인으로 연결. 내부 상태를 외부에서 읽고 쓸 수 있게 함.

Setup Time (t_s): 클럭 엣지 이전에 D 입력이 안정적이어야 하는 최소 시간.

Shannon Expansion: f(a,b,c) = a'·f(0,b,c) + a·f(1,b,c). 변수를 기준으로 함수를 두 개로 분리.

Slack: 타이밍 여유. = t_cy - (t_dCQ + t_dMax + t_s). 양수면 통과, 음수면 위반.

One-Hot 인코딩: n개 상태에 n비트를 사용. 한 번에 1비트만 1. next state 수식을 전이도에서 직접 읽을 수 있음. FPGA에 적합.

State Assignment: FSM의 각 상태에 이진수 코드를 부여하는 과정. Binary 또는 One-Hot 방식. 선택에 따라 회로 크기/속도 달라짐.

SoP (Sum of Products): AND항들을 OR로 연결한 불 표현식 형태.

τ (시정수): FF의 메타스태빌리티 해소 속도. 작을수록 빨리 안정됨.

Transparency: Latch의 특성. Enable=1 동안 입력 변화가 즉시 출력에 반영됨.

디지털 추상화 (Chap 1.1)

- 노이즈 마진 내 노이즈 → 게이트 통과 시 자동 복원
- 전압 파라미터 (2.5V LVCMOS): V_OL=0.2V, V_IL=0.7V, V_IH=1.7V, V_OH=2.1V
- Forbidden Zone: V_IL ~ V_IH (입력 미정의 구간)
- DC Transfer Curve: 입력→출력 전압 관계, 신호 복원 조건 만족 필수

Verilog 핵심 (Chap 1.2)

- `wire` + `assign` = Continuous Assignment = 조합 논리 (입력 바뀌면 즉시 반영)
- `reg` + `always @(posedge clk)` = 순차 논리
- `wire f = expr` ≡ `wire f; assign f = expr` ≡ `assign f = expr` (합성 동일)
- Synthesis: Transform (RTL→게이트) → Optimize (제약 만족)
- 합성 가능: assign, always @(*), always @(posedge clk)
- 합성 불가: #(시간 지연), initial, $display

설계 플로우 (Chap 2)

`Spec → System → Architecture → RTL/Logic → HW Implementation → 통합 평가`

- HW/SW partitioning: System 단계
- High-level vs RTL synthesis 차이: clock step scheduling 여부
- Logic 최적화 단계: wire 지연 = 0 가정
- Physical Design 순서: Floorplan → PDN → Placement → CTS → Signal Routing → RC Extraction

불 대수 핵심 정리 (Chap 3.1)

공리: Identity(1·x=x, 0+x=x), Annihilation(0·x=0, 1+x=1), Negation(0'=1, 1'=0)

자주 쓰는 정리: - Absorption: x+(x·y)=x, x·(x+y)=x - Combining: (x·y)+(x·y')=x ← K-map의 핵심 - DeMorgan: (x·y)'=x'+y', (x+y)'=x'·y' ← NAND/NOR 변환 핵심 - 특수 Distributive: x+(y·z)=(x+y)·(x+z) ← 일반 수학과 다름! - Duality: AND↔OR, 0↔1 교환 시 모든 정리 성립

Normal Form & 게이트 변환 (Chap 3.2)

- Canonical SoP = Σm(출력=1인 행 번호) → 유일성 보장
- SoP → 2단 AND-OR → NAND-NAND (드모르간, 기능 동일)
- Bubble Rule: ○○ 쌍은 상쇄 (이중 반전)
- AOI(AND그룹 OR 후 반전), OAI(OR그룹 AND 후 반전) → 트랜지스터 ~50% 감소

- XOR: a⊕b = a'b+ab', a⊕1=a'(제어 반전), a⊕0=a

카르노 맵 (Chap 6.1)

- Gray Code 배열 → 인접 칸 1비트만 다름 → Combining 정리 시각화
- 묶음 규칙: 2의 거듭제곱, 직사각형, 최대한 크게, 가장자리 연결
- PI 찾기 → Essential PI 선택 → 나머지 최소 커버 = 최소 SoP

Latch & Flip-flop 진화 (Chap 10)

```
RS Latch → Gated Latch → Master-Slave FF → D Flip-flop
  SR=11금지  Transparency   1's Catching    현재 표준
```

- D FF 타이밍 파라미터: Tsu(1.8ns), Th(0.5ns), Tc2q(1.1ns)
- FF 전력 큰 이유: ①클럭 스위칭(전체 20~40%) ②내부 노드 매 클럭 충방전

Timing Constraints (Chap 15)

- t_d ≥ t_c (Propagation ≥ Contamination)
- t_dCQ: Clock-to-Q 최악 / t_cCQ: Clock-to-Q 최선
- Setup 위반 → 클럭 주기 늘리기 / Hold 위반 → 버퍼 삽입
- Hold 위반은 클럭 주기와 무관! ("Unsafe at any speed")

Metastability (Chap 28)

- Setup/Hold 위반 시 → 0도 1도 아닌 불법 상태
- DV(t) = DV(0)×e^(t/τ) → 지수적 해소 but 시간 불확실
- P_failure = f_a × f_cy × (t_s+t_h) × exp(-t_w/τ)
- 제거 불가, 확률만 줄이는 것

Synchronizer Design (Chap 29)

- Brute-Force: FF 직렬 연결로 t_w 확보 → P_S 기하급수적 감소
- 멀티비트: Gray Code (1비트만 변화) → Brute-Force 적용 가능
- 임의 멀티비트: FIFO Synchronizer (포인터만 Gray Code + Sync)
- 사용처: 비동기 외부 입력, 클럭 도메인 경계

State Assignment (Chap 14.2)

- FSM 상태에 이진수 코드를 부여하는 과정. 선택에 따라 회로 크기/속도 달라짐
- Binary: ⌈$\log_2 n$⌉ 비트, FF 적음, 조합 복잡. Gray Code 순서로 배정 권장
- One-Hot: n비트, FF = 상태 수, 조합 단순. next_X = OR(X로 전이되는 모든 조건)
- Verilog FSM 3단 구조: ①FF(State Reg) ②always@(*)+casex(Next State) ③assign(Output)
- `define으로 상태 코드 정의 (가독성, 변경 용이)
- One-Hot 사용처: FPGA(FF 풍부), 속도 중요, 상태 적은 FSM

Moore vs Mealy FSM

- Moore: 출력 = f(state), 상태 원 안에 출력 표기, 클럭 동기 안정 출력
- Mealy: 출력 = f(state, input), 전이 화살표에 입력/출력 표기, 즉각 반응
- Moore 장점: 글리치 없음, 디버깅 쉬움 → 더 많이 사용
- Mealy 장점: 1클럭 빠른 반응, 상태 수 적음

- Verilog: Moore는 출력을 별도 assign, Mealy는 next state와 같은 always 블록
- 신호등 FSM = Moore (출력이 상태만으로 고정, carew는 next state에만 영향)

FSM Modeling & Synthesis (추가자료)

정리하기 ★ - FSM 설계에서 가장 중요한 단계: **State Diagram** 정확히 도출 - FSM 구조: **Moore FSM**과 **Mealy FSM**으로 구분 - Synchronous Mealy FSM: Mealy의 단점인 출력이 입력에 **asynchronous**하게 영향받는 것 방지 - **Output-oriented State Assignment** 기법 적용 시 → Moore FSM을 Synchronous Mealy FSM으로 변환 가능

문제 유형 ★ - 문제1: Moore FSM State Transition Diagram 작성 - 예: 매 클럭 1비트 입력, 마지막 3비트 패턴이 101 또는 010이면 OUT=1 (disjoint 3비트 단위) - 문제2: 같은 내용을 Mealy FSM으로 작성 - 문제3: 주어진 Moore STD가 수행하는 동작 파악 - 문제4: Implication Chart Method 적용 (State Minimization)

Multipliers (추가자료)

곱셈 기본 원리 - A × B = Σ (A AND B[i]) << i (Shift-and-Add) - n비트 × n비트 → n^2 개의 1비트 Partial Product - 결과: 2n 비트

Parallel Multiplier 구조 - ① AND 배열: 모든 Ai·Bj 계산 (n^2 AND 게이트) - ② CSA 트리(Compressor): 같은 비트 위치의 부분곱을 3개씩 묶어 압축 → O(log N) 층 - ③ Final Adder(CLA): 마지막 2개 수 합산

Sequential vs Parallel - Sequential: 회로 작음, 시간 많이 걸림 - Parallel: 회로 큼, 시간 적게 걸림

상수 곱셈 최적화 - Shift-and-Add: 1이 적게 분포 시 → (a<<bit위치)들의 합. **변수 입력** 곱셈에 효과적 - Canonical Encoding: 1이 연속 시 상위+1 - 하위로 변환 → 1의 개수 감소. **상수 입력** 곱셈에 효과적 예: a×62 = a×64 - a×2 = (a<<6) - (a<<1) = (a<<6) + ~(a<<1) + 1

FSM 설계 전체 흐름 (Chap 14.3)

```
① 동작 서술 → ② State Diagram → ③ State Table(추상)
→ ④ State Assignment(이진코드) → ⑤ Encoded State Table
→ ⑥ K-map(next state 비트별) → ⑦ Logic Equations → ⑧ 회로도
```

- State Table: 현재상태 | next(!input) | next(input) | 출력
- Encoded Table: 추상 상태명을 이진수로 교체
- K-map: next state 비트(ns1, ns0) 각각, 출력 비트 각각 따로 작성
- Moore 출력 K-map: state 비트만 변수 (입력 열 없음)
- 신호등 Binary(Gray Code) 예시:
  - ns1 = s0
  - ns0 = (s0 | carew) & !s1
  - lgns=!s1&!s0, lyns=s0&!s1, lrns=s1, lgew=s0&s1, lyew=!s0&s1, lrew=!s1
- Gray Code State Assignment 이유: K-map 인접 묶기 쉬움, 글리치 최소화
- Verilog 스타일 가이드: DFF 명시적 인스턴스, Inferred Latch 금지, reset 필수, `define 사용

Low Power Design

- Dynamic Power: $P = ½SCV^2F$ → V^2 민감 (전압 낮추기 최우선)
- Static Power: Leakage → 공정 미세화로 비중 급증
- Clock Gating: 클럭 차단(Dynamic↓), ILD(en 기반)/DTD(XOR 기반)
- Power Gating: 전원 차단(Static+Dynamic↓), SRFF로 상태 보존
- DVS: 전압/주파수 동적 조절, DC-DC Converter 필요

정리하기 — 핵심 요약 (슬라이드 직접 발췌) - Power는 **Static Power(Leakage)**와 **Dynamic Power**로 나뉜다 - Leakage power에 가장 민감한 두 요소: **Vth**, **transistor channel width (W)** - Dynamic power에 가장 민감한 두 요소: **supply voltage (V)**, **switching activity factor (S)** - Power를 줄이는 주요 두 가지 이유: **발열(thermal)**, **배터리 수명** - Clock Gating 두 종류: **ILD** (MUX feedback loop 있는 FF에만 적용), **DTD** (XOR 기반, 범용) - DTD의 단점: **XOR overhead** (면적/지연 증가), 완화 방법 연구 중 - Power Gating: **PDN(전원 공급망)**을 동적으로 차단 → **leakage** 전류 소모 감소 - DVS: **공급 전압(supply voltage)**을 동적으로 변화 → **dynamic** 전류 소모 감소 - Power Gating + DVS: **IoT** 관련 제품에 필수적인 설계 요소 - Power Gating 설계 고려사항: ①**switching cell**에 의한 IR-drop ②**state retention**에 따른 설계 overhead - DVS 추가 overhead: **voltage regulator**의 효율적인 사용 필요

FA Based Adders (Chap 12.1)

- $S = A \oplus B \oplus CI$, $CO = AB + B{\cdot}CI + CI{\cdot}A$
- "(3,2) counter": 3입력 중 1의 개수를 2진수로 출력 (CO=10의 자리, S=1의 자리)

RCA (Ripple-Carry Adder) - FA N개 직렬 연결. Carry가 LSB→MSB로 순차 전파 - Area = O(N), Delay = O(N) — carry chain이 병목 - Adder/Subtractor: sub=0→덧셈, sub=1→뺄셈 (B XOR sub, Cin=sub) - Overflow 검출: Cin(MSB) XOR Cout(MSB) = 1이면 Overflow

CSA (Carry-Save Adder) - CO를 전파하지 않고 저장 → carry chain 없음 - 3개 입력 → (SUM줄, CARRY줄) = Redundant Binary Number로 압축 - Area = O(N), Delay = O(1) ← 핵심 장점 - 마지막에 CLA 한 번으로 최종 합산 - 3개 합: CSA(O(1)) + CLA(O(logN)) vs 일반 CLA 2번 - 뺄셈 처리: a-b-c-d-e = a+b'+c'+d'+e'+4 (비트반전 + 상수 보정) - CSA 주요 목적: **critical path 줄이기**

P&G Based Adders (Chap 12.2)

P&G 기본 정의 - $G_i = A_i{\cdot}B_i$ (Generate: A=B=1이면 무조건 Carry 생성, Cin 불필요) - $P_i = A_i \oplus B_i$ (Propagate: Cin을 Cout으로 전달) - $C_{i+1} = G_i + P_i{\cdot}C_i$, $S_i = P_i \oplus C_i$

CLA (Carry Look-ahead Adder) - Carry를 원래 입력(P,G)에서 직접 전개 → 중간 Carry 대기 없음 - 예: C4 = G3+P3G2+P3P2G1+P3P2P1G0+P3P2P1P0C0 - 2단 AND-OR, Delay=O(1) 이론적 - 단점: Cn의 fanin = 2n+1 → N 커질수록 fanin 급증 → 소규모에만 실용적

Group P&G (블록 결합) - $G(i{:}j) = G(i{:}k) + P(i{:}k){\cdot}G(k{-}1{:}j)$ - $P(i{:}j) = P(i{:}k){\cdot}P(k{-}1{:}j)$ - 블록들을 재귀적으로 결합 → Prefix Tree 구성 가능

PPA (Parallel Prefix Adder) - 3단계: ①Pi,Gi 계산 ②Prefix Tree로 G(i:-1) 계산(=Ci+1) ③Si=Pi⊕Ci - 2단계 구조에 따라 이름이 달라짐 - Kogge-Stone: 빠름(log N), 배선 복잡, 면적 큼 - Brent-Kung: 면적 작음, 약간 느림 - Delay=O(log N), Area=O(N log N) - 이론적으로 N-bit adder delay를 $\log_2 N$에 비례하도록 구현 가능

Carry-Skip Adder - P(i-1:k)=0 → 블록 내에서 Carry 생성 (일반 동작) - P(i-1:k)=1 → Cin이 블록 전체를 Skip (MUX로 우회) - Delay=O(√N), 면적 효율적

Carry-Select Adder - 상위 블록을 Cin=0, Cin=1 두 경우 동시 계산 (중복 하드웨어) - 하위 Carry 확정 시 MUX로 올바른 결과 선택 - Delay=O(√N), Carry-Skip보다 구현 단순

Combinational Building Blocks (Chap 8)

- Decoder: 이진→One-hot. `b = 1<<a`. 분할: n비트를 k비트씩 나눠 Decoder + AND 조합
- Encoder: One-hot→이진. Decoder 역방향. 대형: 2단(Low bits + High bits), any-input-true(c=|a) 필요
- MUX: select 신호로 입력 중 하나 선택. One-hot select: `({k{s[i]}} & ai)` OR. Binary MUX = Decoder + One-hot MUX
- Arbiter: 첫 번째 1 찾아 grant. carry chain: `c = {..., 1'b1}; g = r & c`
- Priority Encoder = Arbiter + Encoder (첫 1의 위치를 이진수로 출력)
- Eq Comparator: `assign eq = (a==b)`. Mag Comparator: `assign gt = (a>b)`
- Maximum Unit: Magnitude Comparator + 2:1 MUX 결합
- Barrel Shifter: 순환 시프트. 밀려난 비트가 반대쪽으로 재투입

ROM (Read-Only Memory) - 인터페이스: n비트 주소(a) → m비트 데이터(d). 읽기 전용, 비휘발성 - 내부 구조: Decoder(주소→Word line One-hot) + 연결망(저장값) + Bit line - 연결 있음=1, 연결 없음=0으로 데이터 저장 - 2D Array: 주소를 상위(행→Decoder)와 하위(열→MUX)로 분할 → 정사각형 배열 → Bit line 길이 최소화 → RC 지연 감소 → 속도↑ → 예: 256×16 ROM = 64행×64열, a[7:2]=행, a[1:0]=열 MUX - 논리 함수 구현: 진리표 그대로 저장 → 임의 함수 구현 가능 (단, 크기 2^n 증가)

RAM (Read-Write Memory) - Single-port: 주소(a)/데이터입력(di)/데이터출력(do)/쓰기신호(wr) 공유 - Dual-port: 읽기주소(ao)와 쓰기주소(ai) 분리 → 읽기+쓰기 동시 수행 가능 - 내부: Read Decoder + Write Decoder + Gated Latch 배열 → 쓰기: Write Decoder 활성화 & wr=1 → Latch 열림 → di 저장 → 읽기: Read Decoder 활성화 → Latch Q → 버퍼 → do 출력 - 용도: FIFO, Register File, Cache 등

PLA (Programmable Logic Array) - 구조: AND 배열(필요한 곱항만 생성) + OR 배열(곱항 선택적 OR) - ROM과 차이: ROM=모든 Minterm(2^n 개) 생성, PLA=필요한 것만 → 면적 효율↑ - 같은 Product Term을 여러 출력에서 공유 가능 → 복수 SoP 함수 동시 구현

MUX/Decoder로 임의 논리 함수 구현 - Decoder: 원하는 Minterm들을 OR → `wire f = b[1]|b[3]|b[5]|b[7]` - MUX: Binary select MUX, 데이터 입력에 진리표 상수 직접 입력

Number Systems (Chap 11)

정수 음수 표현 3가지 - Sign-Magnitude: MSB=부호(0양수/1음수), 나머지=절대값. 0 두 개, 덧셈 복잡 → 미사용 - 1's Complement: 전체 비트 반전. 0 두 개, End-around carry → 미사용 - 2's Complement: 비트 반전 + 1. 0 하나, 덧셈 단순 → 현대 표준

2의 보수 핵심 - 변환: 비트 반전 후 +1 (역변환도 동일) - 뺄셈 = 덧셈: A - B = A + (B의 2의 보수) - 올림수(Carry out) 발생 시 무시 (정상 동작) - Overflow 검출: Carry-in(부호비트) XOR Carry-out(부호비트) = 1 - Overflow 발생 조건: 양수+양수=음수 OR 음수+음수=양수

Fixed-Point (si.f 형식) - i = 정수 비트 수, f = 소수 비트 수, s = 부호 (2의 보수) - 변환: 실수 × 2^f → 반올림 → 이진수 - 절대 오차(최대) = 2^(-(f+1)) - 예: 1.389 → ×8=11.112 → 반올림=11 → 1011_2 = 1.375

Floating-Point (M × 2^(E-Bias)) - Mantissa(가수): 정규화 = 1.xxx 형태, Implied 1 (맨 앞 1 저장 생략) - Exponent(지수): 실제 지수 = 저장값 - Bias - 부호: 부호-절대값 방식 (MSB) - 상대 오차: 값의 크기에 비례, 넓은 범위에서 균일한 정밀도 - IEEE 754 (32비트): 부호 1 + 지수 8 + 가수 23, Bias=127

Verification and Test (Chap 20)

- Spec Coverage: 스펙 기능 100% 검증 필수
- Impl Coverage: 코드 모든 줄 + FSM 모든 엣지 실행
- Directed Test: 경계 케이스 수동 작성 (23:59:59→00:00:00 등)
- Random Test: 무작위 입력, 상위 모델(C++)과 출력 비교
- STA (Static Timing Analysis): 패턴 없이 모든 경로 타이밍 수학적 분석. False Path 오보고 한계
- Formal Verification: 수학적 증명, 패턴 불필요, 소규모 블록 적합

DFT (Design For Testability) 기법 목록 ★

```
목적: 제조된 칩의 내부 상태를 외부에서 제어(Controllability)하고
      관측(Observability)할 수 있게 하여 테스트 품질을 높임

기법 1: Scan Chain
  목적: 순차 논리(FF)를 조합 논리처럼 테스트
  방법: FF들을 직렬 체인으로 연결 → Scan-in으로 원하는 상태 주입 → 1클럭 →
Scan-out으로 결과 읽기
  효과: 모든 FF 상태 직접 제어/관측 가능

기법 2: ATPG (Automatic Test Pattern Generation)
  목적: Stuck-at Fault를 검출하는 테스트 패턴 자동 생성
  방법: Sensitization(결함값의 반대 인가) + Propagation(출력까지 전달)
  효과: 수동 테스트 불필요, Fault Coverage 수치화

기법 3: BIST (Built-In Self Test)
  목적: 칩 스스로 자가진단 → 외부 테스트 장비 불필요
  RAM BIST: March Algorithm (0쓰기→읽기→1쓰기→읽기 순환)
  Logic BIST: LFSR(패턴생성) + Scan + MISR(서명압축) → Pass/Fail

기법 4: Boundary Scan (JTAG)
  목적: 칩 간 연결(PCB 보드) 테스트
  방법: 각 핀에 Scan 셀 추가 → 핀 상태 직렬로 제어/관측
```

Test (제조 테스트) — 제조 후 전수 - Stuck-at Fault: SA0(항상 0 고착)/SA1(항상 1 고착) → 실제 제조 결함과 유사 - Fault 검출 조건: Sensitization(결함값 반대 인가) + Propagation(출력까지 전달) - ATPG: Fault 검출 테스트 패턴 자동 생성. Fault Coverage = 검출/전체 - Redundant Logic: 제거해도 기능 동일 → Undetectable Fault 발생 가능 - Scan: FF 직렬 체인 → 순차 논리를 조합 논리처럼 테스트 - RAM BIST: March Algorithm (0 쓰기→읽기→1 쓰기→읽기 순환) - LBIST: LFSR(패턴 생성) + Scan + MISR(서명 압축) → 자가진단

Characterization (특성화) — 제조 후 샘플만 - 동작 범위: 전원 전압 범위, 클럭 주파수 범위 - Critical Parameters: V-I 곡선, 전력, 지연, 동작 코너 - 가속 수명 시험: NBTI(PMOS 역바이어스 열화), HCI(고전계 전자 포획), 온도

BDD (Binary Decision Diagram) — 정의 + 작성법 + 예제 ★★

정의

```
논리 함수를 트리(DAG) 구조로 표현하는 자료구조
변수 순서를 고정하고 Shannon Expansion을 반복 적용
ROBDD(Reduced Ordered BDD): 중복 제거 → 유일한 표현 보장
```

Shannon Expansion (작성 원리)

```
f(x1, x2, ...) = x1'·f(0, x2, ...) + x1·f(1, x2, ...)

→ 변수 x1을 기준으로 두 하위 함수로 분리
→ 각 하위 함수에 대해 재귀적으로 반복
→ 최종적으로 0 또는 1 리프 노드에 도달
```

BDD 작성 예제: f = AB + C ★

```
변수 순서: A → B → C

Step 1: A로 Shannon Expansion
  f = A'·(0·B + C) + A·(1·B + C)
    = A'·(C)      + A·(B + C)

  A=0: f = C
  A=1: f = B + C

Step 2: A=1 경우를 B로 전개
  B=0: f = C
  B=1: f = 1

Step 3: 트리 구성
         [A]
        /    \
      0/      \1
      [C]     [B]
      / \    /   \
    0/ 1\ 0/    \1
    0   1  [C]   1
           / \
         0/   \1
         0     1

Step 4: ROBDD로 최적화 (동일 서브트리 공유)
         [A]
        /    \
      0/      \1
      [C]←──[B]
      / \    /   \
    0/ 1\ 0/    \1
    0   1  (C공유)  1
```

BDD 특성

```
유일성:  같은 변수 순서면 같은 함수는 항상 같은 BDD
          → 등가 검증에 핵심 (두 회로의 BDD가 같으면 동일 함수)
효율성:  중복 공유(DAG)로 지수 폭발 억제
EDA 활용: 합성, 검증, 최적화 도구에서 내부 함수 표현에 사용
변수 순서의 중요성:
         좋은 순서 → BDD 크기 작음
```

```
          나쁜 순서 → BDD 크기 지수적 증가
```

주요 회로도 (ASCII) ★★

Full Adder (FA)

```
   A ─┬─[XOR]─┬─[XOR]── S
   B ─┘       │  CI ────┘
       └─[AND]─┤
   B ─┬─[AND]──[OR]── CO
   CI─┘
   A ─┬─[AND]──┘
   CI─┘
(간략): CO = AB + B·CI + A·CI
        S  = A ⊕ B ⊕ CI
```

4비트 RCA

```
A[0]B[0]  A[1]B[1]  A[2]B[2]  A[3]B[3]
  ↓↓        ↓↓        ↓↓        ↓↓
Cin→[FA0]→[FA1]→[FA2]→[FA3]→Cout
      ↓      ↓      ↓      ↓
    S[0]   S[1]   S[2]   S[3]
```

2:4 Decoder

```
a1 a0
 │  │
 ├──┼──[AND(a1' a0')]── b0
 ├──┼──[AND(a1' a0 )]── b1
 ├──┼──[AND(a1  a0')]── b2
 └──┴──[AND(a1  a0 )]── b3

Verilog: wire [3:0] b = 1 << a;
```

4:2 Encoder

```
a3 a2 a1 a0
 │  │  │  │
 ├──┴──┘  └──[OR]── b0  (b0 = a3|a1)
 └─────────[OR]── b1  (b1 = a3|a2)
```

4:1 MUX (One-hot select)

```
s = 0001: b = a0
s = 0010: b = a1     b = (s[0]&a0) | (s[1]&a1) | (s[2]&a2) |
(s[3]&a3)
s = 0100: b = a2
s = 1000: b = a3
```

Arbiter (LSB 우선)

```
r[3] r[2] r[1] r[0]
  │    │    │    │
  │    │    │    └──[AND]←c[0]=1  → g[0]
  │    │    │         ↓c[1]=!r[0]
  │    │    └──[AND]←┘           → g[1]
  │    │         ↓c[2]=!r[1]&c[1]
  │    └──[AND]←┘                → g[2]
  │         ↓c[3]
  └──[AND]←┘                     → g[3]
```

```
g = r & c   (c: carry chain)
```

D Flip-Flop 타이밍

```
CLK  ____/‾‾‾‾‾‾\____
          ↑ 클럭 엣지
D    —[tsu]—╳—[th]—
Q              ——[tCQ]——새값
```

Scan Chain

```
일반 모드:        Scan 모드:
D→[FF]→Q    →   SI→[FF]→SO (직렬 체인)

FF1→FF2→FF3→...→FFn
↑                     ↓
Scan-In          Scan-Out
```